Predicting Plant Traits from Images: A Machine Learning Approach to Understanding Climate Adaptation

Salam Haroun Anvi Parikh Aidan Fasano Tiffany He Yuanzou Gao

May 9, 2025

Abstract

This report outlines a project aimed at predicting six key plant traits (stem/wood density, leaf area per leaf dry mass, plant height, seed dry mass, leaf nitrogen content per leaf area, and leaf area) using images from the iNaturalist database and trait data from the TRY database as well as ancillary data such as coordinates. The model leverages advanced deep learning techniques, including ResNet-50, EfficientNetV2, and Vision Transformers (ViTs), within a multi-task learning framework to simultaneously predict multiple traits. This approach helps analyze plant adaptation to climate change by understanding how plants respond to environmental stressors like temperature fluctuations and water scarcity. While the models showed promising results, such as improved generalization with ResNet-50, they also revealed challenges like underfitting and poor performance on validation data. The findings suggest that further model refinement through regularization, hyperparameter tuning, and data augmentation is needed. The results contribute to strategies for climate adaptation in agriculture, forest management, and biodiversity conservation.

Keywords: Plant Traits, Climate Change, Deep Learning, Multi-Task Learning, Remote Sensing

1 Background

1.1 Problem Definition

This project aims to predict six key plant traits (stem/wood density, leaf area per leaf dry mass, plant height, seed dry mass, leaf nitrogen content per leaf area, and leaf area) using plant photographs from the iNaturalist database and trait data from the TRY database, along with ancillary predictors. It is crucial in the context of climate change because it analyzes plant adaptation to climate stressors such as temperature fluctuations, water scarcity, and changing precipitation patterns across diverse ecosystems. By predicting these traits from images, we can track how plants are physically responding to environmental changes, such as how their leaves grow, how much water and nutrients they store, or how tall they get in response to stress. This helps us understand how plants are adapting to climate change over time. By tracking plant responses through images from global regions, the project helps predict future shifts in biodiversity and ecosystem services, including carbon storage and food production. The insights gained inform strategies for climate adaptation in agriculture, forest management, and biodiversity conservation. This global database aids in understanding how plants cope with environmental changes, supporting efforts for climate adaptation and ecosystem restoration.

1.2 Previous Approaches

To address this problem, we expand on previous research such as Ubbens & Stavness (2017), which used pre-trained CNN models for plant phenotyping tasks but handled each trait independently, lacking a multi-task learning framework. Similarly, Raja et al. (2021) and Dobrescu et al. (2020) applied CNNs to predict multiple traits, but their models were trained on a limited range of plant species (lettuce and rosettes). While these models incorporated multi-task learning, they were not generalizable across different plant species. Our approach seeks to fill these gaps by integrating multi-task learning to predict multiple

plant traits simultaneously while ensuring our model is generalizable across various plant species. This will provide a broader and more flexible framework for predicting plant traits from diverse ecosystems and environments.

Past Kaggle submissions, such as the one found in the "PlantTraits2024 - KerasCV Starter Notebook" (https://www.kaggle.com/code/awsaf49/planttraits2024-kerascv-starter-notebook), predominantly used EfficientNetV2 as the CNN for predicting plant traits. While EfficientNetV2 is an effective model known for its efficiency and accuracy, we wanted to innovate by using a deeper CNN, like ResNet-50. ResNet-50's deep residual learning architecture allows for better feature extraction and captures more complex relationships between plant traits, images, and metadata coordinates. By leveraging ResNet-50, we can better utilize the connections between these elements, enhancing our model's ability to predict multiple traits simultaneously. This approach improves the model's ability to generalize across diverse plant species and environmental conditions, providing more accurate and insightful predictions compared to past solutions that relied on less complex models.

2 Our Approach

Our approach combines advanced machine learning techniques, including ResNet-50, EfficientNetV2, and Vision Transformers (ViTs), to predict multiple plant traits from images. What distinguishes our solution is the combination of multi-task learning and ResNet-50. Multi-task learning enables the model to predict several traits simultaneously, unlike past methods that focused on one trait per model. At the same time, ResNet-50's deep residual learning capabilities enhance the model's ability to capture complex relationships between traits, images, and metadata. Together, these innovations allow our approach to make more efficient and accurate predictions, improving the overall performance and generalizability across different plant species and environments.

To ensure our model generalizes well across various conditions, we train it on a large and diverse dataset of plant images from the iNaturalist database. This diversity (covering different ecosystems, environments, and species) ensures that our model performs reliably on a global scale, not just for a narrow set of plants.

Building on a Kaggle competition, we address gaps in existing solutions by incorporating the ResNet-50 architecture, which has been shown to extract deeper features from images, enhancing prediction accuracy. We also incorporated Vision Transformers (ViTs); these were used in other competition solutions, and we wanted to thoroughly compare the results to those of the other 2 models.

3 Methods

3.1 Data

Our dataset comes from the PlantTraits2024 Kaggle competition, which aims to predict six plant traits using multi-modal inputs. The dataset contains approximately 55,000 plant images, each linked to detailed environmental metadata. These images are sourced from the iNaturalist database, which is a global citizen science platform with plant photographs. The corresponding trait information is collected from the TRY database, an authoritative global plant trait database. In addition, various ancillary predictors, such as climatic and soil variables (e.g., temperature, precipitation, and soil nutrients), are provided. The dataset covers diverse geographic regions, making it a challenging yet comprehensive benchmark for plant trait prediction.

3.2 Input and Output

Each model takes two inputs: a plant image, resized and augmented to 224x224 pixels, and a metadata vector containing 169 features (such as soil and climate data). The output is a 6-dimensional vector, with each value representing the predicted measurement of a specific plant trait.

Model Specific Modifications:

EfficientNetV2: The model processes plant images and metadata to predict six key plant traits. It uses a pre-trained EfficientNetV2 model (frozen to prevent further training) to extract features from the input

images, while metadata is processed separately through a small Multilayer Perceptron (MLP). These two outputs (image features and metadata features) are concatenated into a single vector, which is then passed through two final dense layers. One of these dense layers outputs a 6-dimensional vector representing the predicted means of the plant traits, and the other outputs a 6-dimensional vector for the standard deviations (SDs) of the traits. The model's prediction, when run with model.predict([images, features]), returns a tuple containing both the means and SDs, each with a shape of (batch size, 6), where each of the six values corresponds to a predicted trait for each plant in the batch.

Together, these two outputs describe a probabilistic distribution for each trait, where the mean represents the center of the distribution, and the standard deviation indicates the spread or uncertainty of the prediction. Essentially, the output is not a single point estimate for each trait but a prediction of the distribution of possible trait values, which can be useful for capturing the variability or uncertainty in the model's predictions.

To compare with the other models and for use in evaluation metrics, we used the mean output as the prediction value.

3.3 Data Processing

Explain preprocessing steps:

- **Cleaning:** Missing values in the metadata were imputed using mean imputation for numerical features. Extreme outliers in the target traits (above the 99th percentile) were removed to prevent skewed learning.
- **Splitting:** The dataset was then split into training and validation sets using stratified sampling to maintain balanced distributions of the target traits in both sets.
- **Image Transformations:** All input images were resized to 224×224 pixels and normalized using the ImageNet mean and standard deviation. Data augmentation included random horizontal flips, rotations, and brightness adjustments to improve model robustness.
- **Label Construction:** The target labels include six plant traits: X4 (Stem/Wood Density), X11 (Leaf Area per Leaf Dry Mass), X18 (Plant Height), X26 (Seed Dry Mass), X50 (Leaf Nitrogen Content per Leaf Area), and X3112 (Leaf Area).
  Model Specific Processing:
  CNN (ResNet-50): Only 10% of the total training data was used for model training and validation, selected randomly with a fixed random seed. This sampled data was then further divided into training (80%) and validation (20%) sets, also chosen randomly.
  EfficientNetV2: Only 20% of the cleaned data was randomly sampled using a fixed seed (42) for reproducibility. This sampled data was then split into training (80%) and validation (20%) sets using stratified sampling to ensure a balanced distribution of the target traits across both sets.
  Vision Transformer 50% of the cleaned data was used for training, which was split using stratified sampling. The selected data was split 90% training 10% validation(around 2.7k data points)

3.4 Modeling Techniques

In this project, we explored multiple deep learning architectures to predict six plant traits from both image and metadata inputs. Specifically, we implemented and compared the following baseline models:

- **CNN (ResNet-50):** A convolutional neural network using the ResNet-50 to extract image features, combined with a metadata branch. Our model uses a ResNet-50 backbone, removing its final layer to extract 2048-dimensional features from the plant images. Additionally, a small neural network (MLP) processes the 169-dimensional metadata, reducing it to 64 features. The image and metadata features are then combined (concatenated) and passed through a few more layers to generate the final predictions for the six plant traits.
- **Vision Transformer (ViT):** A deep learning model for image recognition that uses multi-head self-attention, a key component of the transformer architecture, to process image patches as sequences and capture long-range dependencies.

The architecture is based on the "ViT_base_patch16_224" model, pre-trained on ImageNet. The base model architecture was kept unchanged, with the main modification being the addition of the metadata input, which is concatenated in a fusion head before feeding into the model.
The model takes two inputs: an image tensor, processed through the "ViT_base_patch16_224" architecture pre-trained on ImageNet, and metadata, which are processed through separate heads. These inputs are later combined in a fusion head before passing through the final layers for prediction.

- **EfficientNetV2:** This model leverages the EfficientNetV2 architecture. It is pre-trained on ImageNet. A separate branch processes the tabular features, which are concatenated with the image features. Similar to ResNet-50, it integrates both image and metadata inputs via separate branches. The model takes two inputs: an image tensor and a feature tensor. These are processed through separate branches and concatenated before being passed to the output layers.

3.5 Hyperparameter Tuning

- **CNN (ResNet-50):** Hyperparameters were tuned using a combination of strategies: the learning rate was halved every 2 epochs to aid model convergence, the Adam optimizer was used with default settings for its hyperparameters (such as learning rate, 1, 2), and the model was trained for 10 epochs and selected based on previous experimentation to achieve optimal performance. Validation was used during hyperparameter tuning to monitor model performance and prevent overfitting.

  EfficientNetV2: The dropout rate was reduced to 0.1 to address underfitting, and the learning rate was increased to 3e-4 for faster convergence. Batch size was adjusted for optimal performance and stability. Validation set was also used to aid in hyperparameter tuning.

- **Vision Transformer (ViT):** The learning rate was set to 1e-3 and halved every 2 epochs, with training conducted over 10 epochs. The model was validated on the 10% of the data reserved for validation (around 2.7k data points), and this helped tune the hyperparameter as well.

3.6 Evaluation

- These metrics are well-suited for regression tasks as they are designed to evaluate continuous outputs, such as plant trait predictions. L1 loss and MSE measure the difference between the predicted and actual continuous values, which is crucial for regression, where the goal is to minimize this error. Additionally, R^2 loss indicates the proportion of variance in the plant traits captured by the model, providing insight into the model's explanatory power, which is also essential for effective regression analysis.
  CNN (ResNet-50): The model was evaluated using three metrics: L1 loss (mean absolute error) to measure the average magnitude of prediction errors, MSE (mean squared error) to penalize larger errors more heavily, and R^2 (coefficient of determination) to assess how well the model explains the variance in the plant traits. These metrics were chosen to provide a comprehensive understanding of the model's accuracy, sensitivity to large errors, and overall predictive performance.
  EfficientNetV2: The model was trained using a custom R^2-based loss function (R2Loss), optimizing the R^2 score directly instead of traditional loss functions like MSE. The loss was applied to both the main and auxiliary outputs, with weights of 1.0 and 0.3 respectively.
  The primary evaluation metric was the R^2 score on the main output head, with MSE and MAE also tracked to provide additional insights into model accuracy and convergence.
  Together, these metrics provide a clear understanding of the model's performance. R^2 shows how well the model captures the variability in plant traits, while MSE and MAE give deeper insight into the magnitude and consistency of prediction errors, ensuring the model is both accurate and robust for real-world applications.
  Vision Transformer (ViT): The model's objective function was to minimize Mean Squared Error (MSE). It was evaluated using two metrics: MSE Loss, to measure the average squared error between predicted and actual values, and R^2, to measure how well the model explains the variance in the plant traits.

4 Results

4.1 Findings

The tables show the average loss calculated across 6 predicted plant traits. Each plant trait prediction is a continuous variable, allowing us to compute the average loss as the validation metric to compare and analyze.

As shown in tables 1 and 2, the highest performing model was the ResNet 50 CNN, with a training $R^2 = 0.31$ and validation $R^2 = 0.13$. The EfficientNet performs just above baseline with an $R^2 = 0.03$ and validation $R^2 = 0.04$. While both CNNs were able to capture patterns to perform above baseline, the ResNet does perform significantly better in R^2, the metric which this competition is scored in. Finally, the Vision Transformer performed the worse, with a training and validation R^2 below the baseline. While the MSE is comparable to the other 2 models, it does not capture the same amount of variance as the CNN models.

Table 1: Average Training Loss Across Traits

Model	**Loss**	$\mathbf{R^2}$
CNN (EfficientNet V2)	73540	0.0383
ViT	704848	-0.05
CNN (ResNet 50).v2	648540	0.3189

Table 2: Average Validation Loss Across Traits

Model	**MSE Loss**	$\mathbf{R^2}$
CNN (EfficientNet V2)	77210.7031	0.0463
ViT	949781	-0.22
CNN (ResNet 50).v2	1097735	0.1303

4.2 ResNet50

For our model evaluation, we focused on both overall metrics and per-trait performance to better understand how well the model is performing.

In terms of overall performance, the R^2 score for the training set was around 0.32, while for the validation set, it was about 0.13. This suggests that the model can explain some of the variance in the training data, but it struggles to generalize to unseen validation data. The drop in performance from the training set to the validation set indicates that the model is likely overfitting. Overfitting occurs when the model memorizes the training data rather than learning patterns that generalize well to new data.

We also observed the overall loss metrics (MAE and MSE) for both the training and validation sets. The gap between training and validation losses reinforces the idea that the model may not be generalizing well. The model performs well on the training data but struggles with new, unseen data.

The graph of loss over epochs provides more insight into the model's behavior. We noticed that the training loss kept decreasing steadily, indicating that the model was learning from the training data. However, after around epoch 3, the validation loss started to increase. This is a typical sign of overfitting, where the model starts memorizing the training data and performs poorly on unseen data.

To address overfitting, we could explore stronger data augmentation techniques (e.g., rotations, flips, and other transformations). This would help the model learn more generalized features and reduce the likelihood of overfitting.

Our current hyperparameters, such as learning rate and batch size, may not be optimal. Fine-tuning these hyperparameters could help strike a better balance between training speed and model performance, potentially improving generalization. We would have to change the parameters and rerun. Although retraining the model to find the best parameter is computationally expensive, it is worth a try to achieve better performance.

Since some traits are much harder to predict, we could consider training separate models for each trait. This approach would allow each model to focus on the specific patterns and complexities of each trait, potentially improving accuracy for difficult-to-predict traits. Although the novelty in our approach lies in the multi-task learning, it could be interesting to note that individual models made for each trait could perform better.

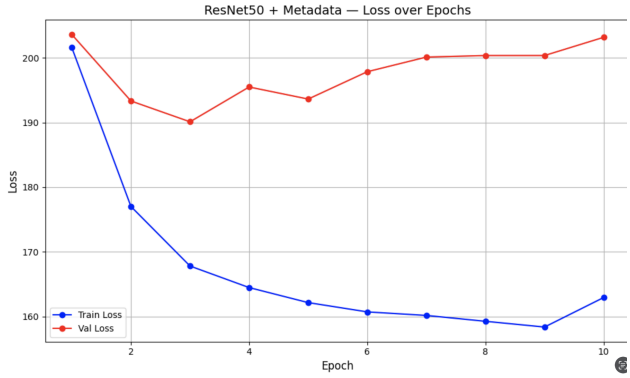

Figure 1: ResNet 50 L1 Loss

4.3 EfficientNet

The EfficientNetV2 model was trained for 20 epochs on a 20% stratified subsample. During training, the mean squared error (MSE) on the training fold decreased from approximately 84800 to 73540, while the validation MSE declined from about 80230 to 77210. The closely parallel trajectories of training and validation error indicate that the model remains under-fit rather than over-fit. Likewise, the coefficient of determination (R^2) on the main output head rose steadily from near zero to 0.048 on the training set and to 0.050 on the validation set, demonstrating progressive improvements in explained variance.

This relatively low MSE can be attributed to two factors. First, the use of a frozen, ImageNet-pretrained EfficientNetV2 backbone supplies robust, generalizable image features without additional tuning. Second, the

multi-task head—jointly predicting trait means and standard deviations—serves as an implicit regularizer, encouraging the network to learn representations that capture both central tendency and variability. In response to initial under-fitting, dropout was reduced from 0.2 to 0.1 and the learning rate was increased from 1e-4 to 3e-4; however, the continued, nearly parallel decline in training and validation loss suggests further capacity remains untapped.

To improve performance, we will unfreeze and fine-tune the upper backbone layers after an initial warm-up, train on the full dataset instead of a subset, extend training to 30–50 epochs, and adopt a cosine learning-rate schedule for more precise optimization. After unfreezing, we will also introduce weight decay or modestly increase dropout to mitigate overfitting while fully leveraging the model's capacity.

Figure 2: EfficientNet V2 Loss

4.4 Vision Transformer

Throughout 10 epochs, the Vision Transformer struggled with stability. As shown in Figure 3, the Validation Loss and R^2 were unusually volatile. This may have been from a learning rate that was too high, or possibly from too small of a validation split. The Training R^2 converged around 0, and while the validation R^2 also reached a minimum of around 0, it decreased for the last epoch. The training loss steadily decreased to around 750000, however the validation loss throughout every epoch remained volatile, making it difficult to judge how the model was generalizing.

In general, ViTs benefit from a large amount of data. Only training it on half of the data for 10 epochs is not the optimal setup for the highest performing outcome. In the future, having access to a powerful GPU would allow this model to train on the full dataset, at a lower learning rate, for more than 10 epochs, which would have allowed it to learn more effectively. In theory, due to the ViT's ability to see relationships through all patches at once, it would have been able to recognize patterns that the CNN's may have missed. However, due to the limitations of this research, we could not accurately measure its capability to the fullest extent.

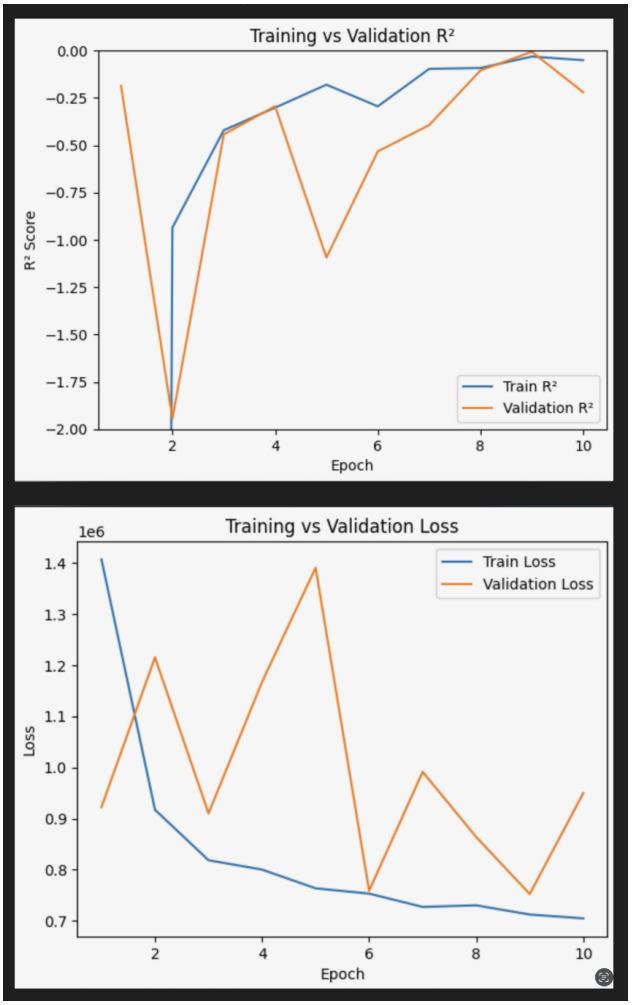

Figure 3: ViT Loss

5 Discussion

5.1 Interpretation of Results

Our best model, ResNet50, achieved a R^2 of 0.13. This is a modest result, but above the baseline. This suggests that Vision features do carry importance regarding key traits (e.g., leaf area). Being able to infer trait distributions impacts climate modeling positively, opening the door to defining parameters for ecological processes like carbon assimilation or water use. However, the performance of our models has much room for growth before we can trust it to produce real-world impact. The underperformance of EfficientNetV2 and ViT suggests that deeper residual networks may be better at capturing plant structure. There is also a possibility that the data has limitations when working with these particular models. We note that computer vision can serve as a tool for predicting and monitoring plant traits, but we highlight the need for model refinement.

5.2 Future Work

Our proposal of next steps:

- Incorporate more metadata. We could integrate high-resolution soil maps (texture, moisture, organic carbon) and satellite-derived indices (surface temperature, canopy moisture), additional layers that may

help us provide more context on what shapes plant trait expression. In this example, this additional metadata may help us boost our ability to predict water-use traits and nutrient-related traits.

- Refine model architecture–exploring hybrid ConvNet–attention blocks that fuse the strong local feature extraction of CNNs with lightweight global context modules. Another aim is to experiment with tuning head architectures such as deeper MLPs or auxiliary decoders for each trait.
- Transfer learning on new species. To test generalization, likely after refining our model, we could fine-tune it on biological categories that we will exclude from our training data. By adapting only the final layers, we can evaluate how well learned representations transfer to previously unseen species.
- A classification task where we separate the plant from the background. Pre-segmentation of the plant could drastically reduce noise. We propose a U-Net or DeepLabV3 classifier to generate plant masks, then feed the resulting cleaned version into a CNN. This would improve focus on relevant morphology, reducing unnecessary noise.

5.3 Comparison with Previous Literature

We compare our image-based CNN approach alongside several lines of work: (1) tabular, tree-based models from the winning Kaggle entry **Dysdsyd2024**,(2) deep learning and citizen science enabling automated plant trait predictions from photos **Schiller2021**, (3) Other Kaggle submissions

Tabular / Tree-based Pipelines The first-place Kaggle solution (`dysdsyd/PlantTraits2024`) preprocesses site, climate, soil and trait metadata into a one-hot– and mean-imputed feature matrix, then trains LightGBM ensembles to predict each trait. They report cross-validated R^2 values in the range 0.30–0.50 on held-out folds. In comparison, our pure-image ResNet-50 achieved a validation R^2 of 0.13—lower than the tabular tree models—but our pipeline requires only standard RGB photographs and no extensive environmental metadata. Thus, while tree-based models hold the lead in accuracy, our approach offers a more low-cost alternative for large-scale trait mapping where metadata are unavailable or incomplete. However, our model requires significant refining to achieve better performance as well as testing and may not outperform tree-based models in this situation.

Automated Plant Trait Predictions Research Schiller (2021) demonstrated that CNNs trained on heterogenous, citizen-science RGB photos can predict plant traits by learning visible visual plant features. They used ensembles of architectures (Inception-ResNet, Xception, MobileNetV2), and we used similar architectures for our predictions. Schiller et al. further showed that incorporating species-level trait plasticity and WorldClim bioclimatic priors boosts explained variance by up to 16 percentage-points, achieving per-trait R^2 as high as 0.58 for growth height and 0.45 for leaf area. In contrast, our best model, the ResNet50, yielded a validation R^2 of 0.13 across six mean traits. Building on this study, we could boost our ResNet by fusing a species-embedding vector into our regression head. We could also experiment with embedding diverse architectures in an ensemble method.

Other Kaggle Submissions A third notebook submission—using a hybrid CNN+metadata fusion—achieved intermediate performance ($R^2 \approx 0.25$) by concatenating ResNet image embeddings with standardized environmental features before a joint regression head. This result suggests that combining visual and tabular signals can close some of the gap to pure-metadata models, and motivates us to explore multimodal architectures.

6 References

References

[1] Hugging Face. (n.d.). *PyTorch Image Models*. Retrieved from https://github.com/huggingface/pytorch-image-models/blob/main/README.md

[2] PPRP. (n.d.). *Vision Transformer Models*. Retrieved from https://pprp.github.io/timm/models/vision-transformer/

[3] PyTorch. (n.d.). *PyTorch Documentation*. Retrieved from https://docs.pytorch.org/docs/stable/index.html

[4] Kaggle. (2024). *PlantTraits2024 Competition Dataset*. Retrieved from https://www.kaggle.com/competitions/planttraits2024/data?select=test.csv

[5] Awsaf. (2024). *PlantTraits2024 - KerasCV Starter Notebook*. Retrieved from https://www.kaggle.com/code/awsaf49/planttraits2024-kerascv-starter-notebook

[6] Duan, W., et al. (2017). *Plant Phenotyping: Past, Present, and Future*. Frontiers in Plant Science, 8, 1190. Retrieved from https://www.frontiersin.org/journals/plant-science/articles/10.3389/fpls.2017.01190/full

[7] Zhang, Q., et al. (2021). *Efficient Vision Transformer*. arXiv:2112.03205. Retrieved from https://arxiv.org/abs/2112.03205

[8] Duan, W., et al. (2020). *Plant Phenotyping Using Machine Learning*. Frontiers in Plant Science, 11, 141. Retrieved from https://www.frontiersin.org/journals/plant-science/articles/10.3389/fpls.2020.00141/full

[9] Palomino Cobo, F. (2021). *Demystifying Vision Transformers with PyTorch: Understanding Patch Embeddings (Part 1/3)*. Medium. Retrieved from https://medium.com/@fernandopalominocobo/demystifying-visual-transformers-with-pytorch-understanding-patch-embeddings-part-1-3-ba380f2aa37f

[10] Medium. (2021). *EfficientNetV2: Faster, Smaller, and Higher Accuracy than Vision Transformers*. Medium. Retrieved from https://medium.com/data-science/efficientnetv2-faster-smaller-and-higher-accuracy-than-vision-transformers-98e23587bf04

[11] Dosovitskiy, A., et al. (2021). *Vision Transformers: From EfficientNetV2 to Transformers*. arXiv:2104.00298v3. Retrieved from https://arxiv.org/abs/2104.00298v3

[12] Schiller, C., Schmidtlein, S., Boonman, C., Moreno-Martínez, A., & Kattenborn, T. (2021). *Deep learning and citizen science enable automated plant trait predictions from photographs*. Scientific Reports, 11(1), 16395. doi:10.1038/s41598-021-95616-0. Retrieved from https://doi.org/10.1038/s41598-021-95616-0

[13] Kaggle Community. (2024). *PlantTraits2024 Competition Discussion, Post #510188*. Retrieved from https://www.kaggle.com/competitions/planttraits2024/discussion/510188

[14] Korotas, K. (2024). *9th-Place-PlantTraits2024-DinoV2-CatBoost*. Kaggle Code. Retrieved from https://www.kaggle.com/code/korotas/9th-place-planttraits2024-dinov2-catboost

7 Author Contributions

- Salam: Presentation outline, Background, Abstract, Methods, Results
- Anvi: Built pre-processing code for models, Discussion, Presentation outline
- Aidan: Built/Ran ViT model, Ran original ResNet 50 (did not include as it was improved upon). Wrote the Findings and Vision Transformer results pages.
- Tiffany: Built/Ran the EfficientNetV2 model. Wrote the EfficientNet result section in the report.
- Yuanzou: Built/Ran the ResNet 50 L1 Loss version. Wrote the ResNet50-related part in the report.